Projeto da Disciplina: Visão Computacional

- **Aluno:** Anderson Corrêa
- **Curso:** Deep Learning and Vision
- **Disciplina:** Computer Vision (INFNET 26E3_3)

Como ler este relatório

- Cada atividade tem uma seção com: problema, decisões técnicas, resultados e análise crítica.
- Os números e gráficos vêm da execução dos notebooks no Google Colab com GPU T4.
- Cada notebook roda do início ao fim sem alterações e baixa os próprios dados.

Atividade 3: Classificador com CNN pré-treinada

Problema

- Tarefa: dizer a qual de 7 classes pertence uma foto.
- Dataset: `pavansanagapati/images-dataset`, público no Kaggle.
- Classes: bike, cars, cats, dogs, flowers, horses, human. A classe é o nome da pasta da imagem.

O download traz o dataset repetido

- O arquivo do Kaggle tem 3.606 imagens em duas árvores de pastas com o mesmo conteúdo: `data/<classe>/` e `data/data/<classe>/`, com 1.803 arquivos cada.
- Comparando o conteúdo dos arquivos (SHA-256), existem 1.764 imagens distintas.
- 1.725 imagens aparecem 2 vezes, uma em cada árvore.
- 39 imagens aparecem 4 vezes, porque a mesma foto está guardada duas vezes dentro de cada árvore: 1 em bike, 19 em horses e 19 em human.
- Nenhuma imagem aparece em duas classes diferentes.
- O notebook remove as cópias exatas antes de dividir os dados: 1.842 arquivos removidos, 1.764 imagens usadas.
- Formatos dos 3.606 arquivos: 1.616 JPG, 1.570 BMP e 420 PNG.

Imagens distintas por classe

Classe	Imagens
bike	364
cars	420
cats	202
dogs	202
flowers	210
horses	183
human	183

Classe	Imagens
Total	1.764

Divisão estratificada, seed 42

Conjunto	Imagens	Parte
Treino	1.234	70%
Validação	265	15%
Teste	265	15%

Decisões técnicas

- Modelo carregado: ResNet-50 do torchvision com os pesos `IMAGENET1K_V2`, pré-treinados na ImageNet.
- Classification head trocada: a camada final `fc`, que tinha 1.000 saídas, deu lugar a uma camada linear nova com 7 saídas, uma por classe do dataset.
- Feature extraction: todos os pesos do backbone ficam congelados (`requires_grad = False`). Só a camada nova recebe gradiente.
- Pré-processamento igual ao publicado para esses pesos: redimensionar o lado menor para 232, recorte central de 224x224, média e desvio padrão da ImageNet.
- Sem augmentation nesta execução. As propostas estão na seção "Melhorias propostas".

Por que ResNet-50

- Cabe com folga na T4. O pico de memória alocada pelo PyTorch na GPU foi de 465,8 MB, cerca de 3% dos 15,6 GB da placa.
- É rápida o bastante para o Colab. O notebook inteiro rodou em 108,2 segundos, contando download, 10 épocas e avaliação.
- A saída do backbone tem 2.048 features por imagem. Para separar 7 classes, a camada nova precisa de 2.048 x 7 pesos mais 7 vieses: 14.343 parâmetros.
- A ImageNet já contém fotos muito parecidas com as deste dataset: várias raças de cães e de gatos, tipos de carro, bicicletas, cavalos e flores. Pessoas aparecem em muitas fotos da ImageNet, embora ela não tenha uma classe "pessoa".
- Com 7 classes de objetos do dia a dia, um modelo maior não teria erro de teste para corrigir. A validação errou 2 imagens em 265.
- Comparação com a EfficientNet-B0, segundo a documentação do torchvision:
  - A EfficientNet-B0 é menor: cerca de 5,3 milhões de parâmetros e 1.280 features, com 77,7% de acurácia top-1 na ImageNet.
  - A ResNet-50 com `IMAGENET1K_V2` tem 80,9% de top-1. Como a memória e o tempo ficaram longe do limite da T4, a economia da EfficientNet-B0 não faria diferença aqui. Ela seria a escolha se o limite fosse memória ou tempo.

Por que feature extraction

- O dataset é pequeno: 1.234 imagens de treino.
- O domínio é próximo do da ImageNet: fotos comuns de animais, veículos, flores e pessoas.
- A rede tem 23.522.375 parâmetros. Treinar todos com 1.234 imagens daria cerca de 19 mil parâmetros por imagem, com risco alto de overfitting.
- Com o backbone congelado, só 14.343 parâmetros treinam: 0,06% do total.

- O treino fica barato. O backbone só faz a passagem direta e não guarda gradientes, o que explica o pico de memória baixo.

Hiperparâmetros

- **Otimizador: Adam.** Ajusta o passo de cada peso sozinho. Para uma única camada linear, funciona bem sem ajuste fino.
- **Taxa de aprendizado: 0,001.** É o valor padrão do Adam. A perda caiu em todas as épocas, sem oscilar, então não houve motivo para mudar.
- **Épocas: 10.** A acurácia de validação parou de mudar na época 2. As 10 épocas servem para mostrar a curva estável e custam pouco tempo.
- **Batch size: 32.** Dá 39 passos por época e usa pouca memória.
- **Backbone em modo de avaliação.** Em modo de treino, as camadas de batch normalization atualizam suas estatísticas mesmo com os pesos congelados. Em modo de avaliação, o backbone produz as mesmas features no treino, na validação e no teste.
- **Função de perda: entropia cruzada**, sem pesos por classe. A maior classe (cars, 420) tem 2,3 vezes o tamanho da menor (horses e human, 183), um desbalanceamento moderado.
- **Seed: 42**, usada na divisão dos dados e na inicialização da camada nova.

Resultados

Acurácia global no teste: 100% (265 acertos em 265 imagens).

Acurácia por classe no teste

Classe	Imagens no teste	Acertos	Acurácia	Queda com 1 erro
bike	54	54	100%	1,85 ponto
cars	63	63	100%	1,59 ponto
cats	30	30	100%	3,33 pontos
dogs	31	31	100%	3,23 pontos
flowers	32	32	100%	3,13 pontos
horses	28	28	100%	3,57 pontos
human	27	27	100%	3,70 pontos
Global	265	265	100%	0,38 ponto

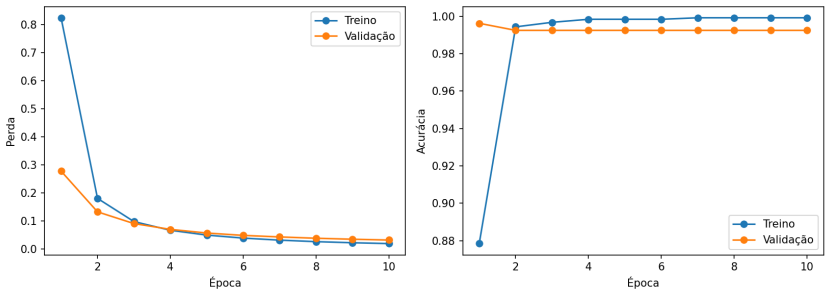
- A última coluna mostra quantos pontos percentuais a acurácia perderia com um único erro.

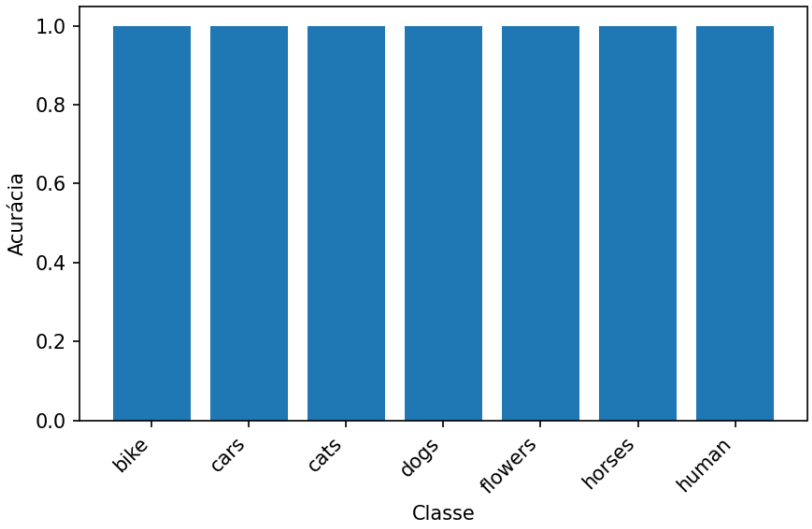
Curvas de treino

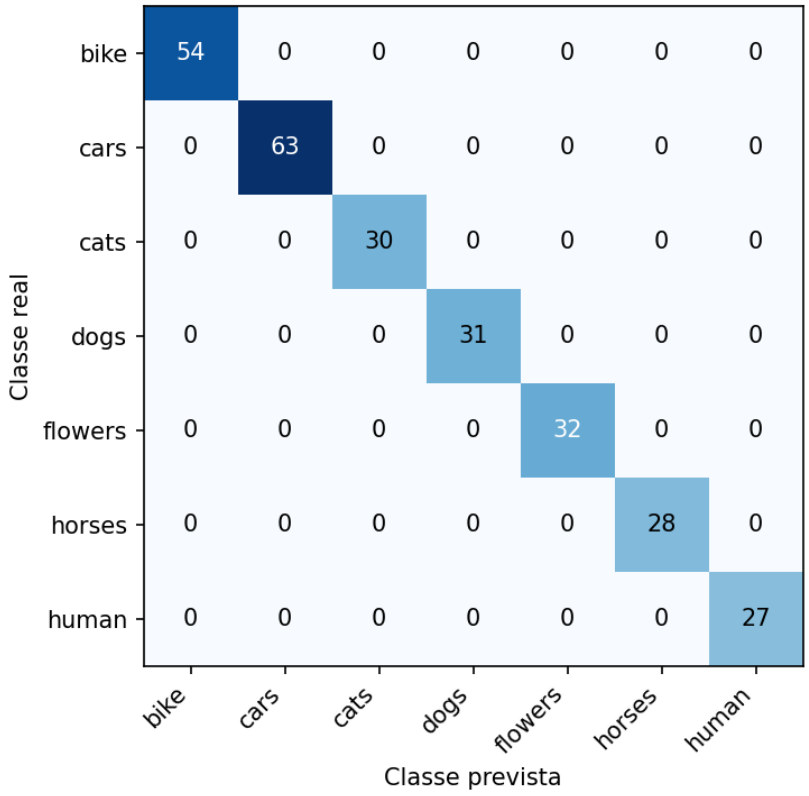
Época	Perda treino	Perda validação	Acurácia treino	Acurácia validação
1	0,824	0,277	87,84%	99,62%
2	0,180	0,132	99,43%	99,25%
5	0,049	0,056	99,84%	99,25%
10	0,018	0,031	99,92%	99,25%

- A acurácia de validação estabiliza na época 2, em 99,25% (263 de 265), e fica igual até a época 10.

- Na época 1 a validação acertou 264 de 265 (99,62%). Da época 2 em diante, 263.
- A perda de validação cai em todas as épocas, de 0,277 para 0,031. Não há subida que indique overfitting.
- Diferença final entre treino e validação: 0,67 ponto percentual na acurácia (99,92% contra 99,25%) e 0,013 na perda.
- Na época 1 o treino aparece pior que a validação (87,84% contra 99,62%). O número de treino é a média durante a época, enquanto a camada ainda está aprendendo. A validação é medida no fim da época.
- No treino, a última época errou 1 imagem em 1.234.







Análise crítica

O que a matriz de confusão mostra

- A matriz de confusão do teste não tem nenhum erro. Não existe par de classes confundido para analisar.
- Os únicos erros medidos estão na validação: 2 imagens em 265.
- O notebook não registra quais são essas 2 imagens nem a classe delas. Sem isso, não dá para apontar a causa. Guardar as previsões da validação é uma melhoria simples para uma próxima execução.

Por que a tarefa é fácil para as features da ImageNet

- As 7 classes são bem diferentes entre si e quase todas têm classes parecidas na ImageNet.
- O backbone já separa esses objetos antes de qualquer treino. A camada nova só precisa aprender a qual das 7 saídas cada grupo de features pertence. Isso explica a validação acima de 99% já na primeira época.
- As classes também diferem no tipo de foto:
  - bike e cars são fotos de rua, quase todas com 640x480 pixels, em BMP.
  - flowers são closes de 128x128 pixels, em PNG, com a flor ocupando o quadro inteiro.
  - cats e dogs são fotos caseiras e de abrigo, em JPG.
  - horses e human são imagens pequenas da internet, em JPG. Em horses há desenhos e imagens geradas por computador. Em human, as pessoas usam roupa de equitação e muitas aparecem ao lado de um cavalo.
- Risco que isso traz: a classe anda junto com o formato e a resolução do arquivo. Uma flor de 128 pixels ampliada para 232 fica borrada. Uma foto de rua reduzida fica nítida. O modelo pode usar essa diferença como atalho. Esse efeito não foi medido.

O primeiro resultado foi descartado

- A primeira execução usou as 3.606 imagens, sem remover cópias, e chegou a 100% no teste.
- Nela, cópias da mesma foto caíam no treino e no teste. O modelo era avaliado em fotos que já tinha visto.

- Esse resultado foi descartado como inválido. Os números deste relatório vêm da execução com 1.764 imagens distintas.
- Limite da correção: só cópias exatas foram removidas. Quase-duplicatas, como a mesma foto redimensionada ou regravada com outra compressão, não foram procuradas. Se existirem, ainda pode haver vazamento entre treino e teste.

O que o resultado não mostra

- Foi uma única execução, com uma única seed e uma única divisão dos dados. A variação entre execuções não foi medida.
- O teste tem 265 imagens. Um erro tira 0,38 ponto da acurácia global.
- As classes do teste têm entre 27 e 63 imagens. Um erro em human (27 imagens) baixaria a classe para 96,30%. Um erro em cars (63 imagens) baixaria a classe para 98,41%.
- Com 0 erros em 265 imagens, o limite superior para a taxa de erro real é de cerca de 3/265 = 1,1%, com 95% de confiança (aproximação conhecida como rule of three). O resultado é compatível com uma acurácia real perto de 99%, como a da validação (99,25%).
- 100% em um teste pequeno indica que o modelo erra pouco neste dataset. Erros continuam possíveis, como os 2 da validação.
- O teste vem da mesma coleção do treino. O resultado não diz como o modelo se sairia em fotos de outra origem, por exemplo carros fotografados de perto ou pessoas com outro tipo de roupa.
- As imagens de human quase sempre mostram roupa de equitação e cenário de hípica. O modelo pode ter aprendido o cenário. O teste não separa isso da figura humana em geral.

Melhorias propostas

- Nenhuma das estratégias abaixo foi executada. A acurácia de teste já está no teto, então o ganho esperado é em fotos de outra origem. Neste teste não há erro para corrigir.
- Todas valem só para o conjunto de treino. Validação e teste continuam com o pré-processamento fixo.

Geometric augmentation

- **Flip horizontal: seguro para as 7 classes.** Nenhuma classe depende de esquerda ou direita.
  - Ajuda mais em bike, cars e horses, que aparecem muito de perfil, virados para um lado ou para o outro.
  - Em human, algumas fotos têm marca d'água com texto. O texto fica espelhado, mas ele não define a classe.
- **Rotação pequena, até 15 graus: ajuda cats e dogs.** Os animais aparecem deitados, no colo e em ângulos variados, e a câmera é de mão.
- **Rotação livre e flip vertical: só para flowers.** As flores são fotografadas de cima e de perto, sem um "lado de cima" definido.
- **Flip vertical e rotação grande distorcem bike, cars, horses e human.** Nessas fotos o chão está sempre embaixo, e as pessoas de human estão de pé. Uma rua de cabeça para baixo não existe no uso real.

Color augmentation

- **Brilho e contraste: ajudam bike e cars.** São fotos de rua com sol, sombra e algumas cenas noturnas com flash. A variação de luz já existe nos dados, e a augmentation cobre os casos intermediários.
- **Mudança de matiz (hue): livre para bike, cars e flowers.** Bicicletas e carros têm pintura de qualquer cor. As flores do dataset são vermelhas, cor-de-rosa, roxas, amarelas, brancas e laranja.
- **Mudança de matiz distorce cats, dogs e horses.** A pelagem fica entre branco, preto, marrom, cinza e laranja. Um cavalo verde ou um gato azul não existe. Para essas classes, a mudança de matiz deve ser pequena.

- **Tons de cinza com probabilidade baixa: ajuda horses e human.** Só essas duas classes têm imagens em preto e branco, e são poucas. Em human há também pelo menos uma foto antiga em sépia. Sem augmentation, o modelo pode aprender que "foto sem cor" quer dizer uma dessas duas classes. Aplicar tons de cinza às vezes em todas as classes quebra esse atalho.
- **Tons de cinza prejudica flowers.** A cor forte da pétala é a pista mais clara da classe. Sem cor, sobra só a forma. Por isso a probabilidade deve ser baixa, perto de 10%.

Scale variation

- Estado atual: o recorte central corta as bordas.
  - Em uma foto de 640x480 (bike e cars), o recorte de 224x224 mantém 72% da largura.
  - Em um retrato de human de 183x275, mantém 64% da altura. Cabeça e botas podem ficar de fora.
- **Recorte aleatório com mudança de escala (RandomResizedCrop): seguro para flowers.** A flor ocupa o quadro inteiro. Qualquer recorte ainda mostra pétalas.
- **Ajuda cats e dogs com recortes moderados.** O animal ocupa boa parte da foto, e a distância da câmera varia bastante.
- **Recorte agressivo prejudica cars e bike.** Em muitas fotos de cars, o carro é pequeno e está longe, em uma rua larga. Um recorte pequeno pode pegar só asfalto, prédio ou árvore, com o rótulo "cars". Como bike e cars têm o mesmo tipo de rua ao fundo, um recorte sem o objeto fica ambíguo entre as duas classes.
- **Recorte agressivo prejudica human.** A pessoa está de corpo inteiro e muitas vezes ao lado de um cavalo. Um recorte pequeno pode mostrar só o cavalo, com o rótulo "human", e empurrar o modelo para confundir human com horses.
- Proposta: recorte com área entre 60% e 100% da imagem, em vez do padrão de 8% a 100%.

Normalização

- **Já usada: média e desvio padrão da ImageNet, por canal.** Vale para as 7 classes e precisa continuar assim. O backbone está congelado, e seus filtros e camadas de batch normalization esperam entradas nessa escala.
- **Média e desvio padrão do próprio dataset: prejudica em feature extraction.** As features sairiam diferentes das do pré-treinamento, e o backbone não pode se ajustar. Só faria sentido com fine-tuning da rede inteira.
- **Normalizar também a resolução: afeta flowers, bike e cars.** flowers tem 128x128 pixels e é ampliada. bike e cars têm 640x480 e são reduzidas. Reduzir todas as imagens para um mesmo tamanho pequeno antes do pré-processamento tiraria a nitidez como pista da classe. É um teste direto para o atalho descrito na análise crítica.
- **Normalização por imagem (cada foto com sua própria média): distorce bike.** As fotos noturnas ficariam com o mesmo brilho médio das fotos de dia, e o ruído do escuro seria ampliado.

Feature extraction ou fine-tuning

Regra de decisão: tamanho do dataset contra distância do domínio em relação à ImageNet

	Domínio próximo da ImageNet	Domínio distante da ImageNet
Dataset pequeno	Feature extraction. Backbone congelado, treina só a classification head.	Caso mais difícil. Fine-tuning só dos últimos blocos, com taxa de aprendizado baixa e augmentation forte.

	Domínio próximo da ImageNet	Domínio distante da ImageNet
Dataset grande	Fine-tuning da rede inteira, com taxa de aprendizado baixa. As features já são boas e os dados permitem refinar.	Fine-tuning da rede inteira. Os pesos pré-treinados servem como ponto de partida.

- **Feature extraction** serve quando as features prontas já separam as classes e há poucos dados. Poucos parâmetros treinam, então o risco de overfitting é baixo e o treino é rápido.
- **Fine-tuning** serve quando as features prontas não bastam. Isso acontece quando o domínio é diferente (raio-X, imagem de satélite, microscopia) ou quando as classes são muito parecidas entre si. Ele exige mais dados, porque milhões de pesos passam a treinar.
- Com poucos dados e domínio distante, as duas opções têm problema: as features prontas servem mal, e treinar muitos pesos leva a overfitting. As primeiras camadas (bordas, texturas) ainda costumam servir, por isso a saída é liberar só os últimos blocos.

Onde este dataset cai

- Tamanho: pequeno. São 1.764 imagens, 1.234 no treino.
- Domínio: próximo. São fotos comuns de objetos que a ImageNet já cobre.
- Célula da tabela: dataset pequeno e domínio próximo. A escolha é feature extraction.
- O resultado é coerente com a escolha: com 14.343 parâmetros treinados, a validação chegou a 99,25% e o teste a 100%.
- Fine-tuning teria pouco a ganhar aqui. Sobram 2 erros na validação e nenhum no teste. Em troca, colocaria 23.522.375 parâmetros para treinar com 1.234 imagens.